Informatica Teorica
Appunti

Giovanni "Qua' Qua' dancer" Palma
e
Alex "Morbidelli WhatsApp" Basta

Contents

Chapter 1 **Introduzione a problemi e indecidibilita** — Page

1.1 Halting Problem
Il Metodo Diagonale — • Dimostrazione Halting Problem —

1.2 Tipi di problemi
Problemi di decisione —

1.3 Linguaggi
Definizioni — • Automi — • Macchia di Turing —

Chapter 2 **Macchine di Turing e Linguaggi** — Page

2.1 Linguaggi
Definizioni — • Automi —

2.2 Macchia di Turing
Definizioni formali — • Esercizi —

2.3
Esercizi — • Macchine Non Deterministiche —

2.4 Macchine di turing universali
Codifica della macchina di turing — • Definizione formale — • Processo di simulazione —

2.5 Problemi Ricorsivi e Ricorsivamente Enumerabili
Enumerare stringhe e macchine — • Vettore caratteristico e argomento diagonale — • Proprietà sui linguaggi —

2.6 Linguaggio Universale e Problema della Fermata
Il complemento di $\mathcal{L}_d$ — • Il Problema della Fermata — • La variante $HALT_\varepsilon$ —

Chapter 3 **Riduzioni** — Page

3.1 Teorema sulle riduzioni

3.2 Esempio: $\mathcal{L}_{ne}$

Chapter 4 **Teorema di Rice** — Page

Chapter 5 **Complessita Strutturale** — Page

Chapter 6 **Complessita Spaziale** — Page

Chapter 1

Introduzione a problemi e indecidibilita

Studieremo due arie:

- **Calcolabilita**: ci chiediamo se per quel problema esistera mai un algoritmo in grado di risolverlo (decidibilita' del problema). Non li incontriamo spesso irl dato che siamo piu' spinti a fare robe che sappiamo gestire. Bisogna studiare la struttura del problema per decidere se esiste o meno un algoritmo che lo risolve.
- **Complessita**: vogliamo determinare se un problema decidibile e' "facile" o "difficile", ovvero qual'e' la sua complessita' (diverso dalla complessita' degli algoritmi)

Note:

Il professore usa in modo informale i termini "facile" e "difficile" perche' si tratta solo di un'introduzione. Piu' avanti definiremo meglio questi concetti in base a se la complessita' e' *polinomiale* o meno.

Definition 1.0.1: Problema

Relazione fra stringhe.

Example 1.0.1 (Somma)

Dati x e y calcola $x + y \to z$. E' una relazione binaria fra stringhe:

- $< (2, 3), 5 >$
- $< (4, 3), 7 >$

  $\vdots$

Dove la prima stringa e' l'input e la seconda e' l'output. Dato che elencare tutte le tuple e' impossibile, le descriviamo a parole ma stando attenti ad essere precisi. Ci servono tre elementi:

- What is input?
- What is output?
- Che relazione c'e' fra out e in?

1.1 Halting Problem

Definiamo prima per bene il problema:

- Input: Una stringa P che rapresenta il codice sorgente di un programma
- Output: Un booleano
- Relazione: Se P termina ritorna T, altrimenti F

Sembra un problema semplice, ma vedremo che in realta' e' indecidibile. Per confermare cio' dobbiamo formulare una dimostrazione formale, usando quel che viene chiamato "metodo diagonale".
Il prof non va cosi' in dettaglio per il metodo usato nella dimostrazione (dato che e' solo un esempio introduttivo), ma secondo me e' interessante e fornisce un bell'esempio per l'utilita' della Teoria delle Categorie.

1.1.1 Il Metodo Diagonale

Il *metodo diagonale* e' uno schema utile per la dimostrazione di certi teoremi. E' stato ideato per la prima volta da Cantor per dimostrare, per esempio, che $\mathbb{R}$ non e' numerabile, dove viene usata la *diagonale* di una tabella per dimostrare un assurdo. Questo metodo e' stato successivamente usato per dimostrare altre proposizioni molto importanti, come il teorema di Cantor (quello sugli insiemi potenza), il paradosso di Russel e i teoremi dell'incompletezza di Godel.
La struttura della dimostrazione generale e' la seguente:

$$\begin{tikzcd}
 & \text{COSE} \times \text{COSE} \arrow[r, "X", "\text{potente}"'] & \text{VALORI} \arrow[d, "I", "\text{Inverti}"'] \\
\text{COSE} \arrow[ur, "D", "\text{Diagonale}"'] \arrow[rr, "S", "\text{Speciale}"'] & & \text{VALORI}
\end{tikzcd}$$

Lo schema e' molto astratto, e' formato solo da *punti* (i vertici) e *frecce* che possono essere composte per formare altre frecce. I primi possono essere insiemi e le frecce funzioni, come vedremo. Le quattro frecce hanno delle funzioni particolari:

- La funzione "potente" X: nella Teoria delle Categorie viene definita *morfismo punto-suriettivo*(?). E' la freccia che assumiamo esista per poi dimostrare l'assurdo.
- La funzione diagonale D: dato un elemento x ritorna la coppia (x, x). Raccoglie l'essenza del metodo diagonale in quanto e' la componente che causa l'*auto-referenzialita'* che sta al cuore della dimostrazione.
- La funzione che inverte I: dato l'output di X lo *inverte* in modo da causare l'assurdita'.
- La funzione "speciale" S: composizione delle tre frecce sopra, insieme all'assunzione iniziale permette di dimostrare l'assurdo.

Note:
Lowkey non l'ho spiegato benissimo dato che non ho studiato abbastanza le sue applicazioni e so solo le basi della Teoria delle Categorie quindi non e' rigoroso per niente. Forse nel futuro ci ritornero', boh.

Finita la costruzione, i prossimi passi sono:

1. Codifica S con una *COSA*, k.
2. Cosa succede se passo k a S?

Note:
In verita' non sono sicurissimo se questi passi fanno parte del metodo generale o solo dell'halting

1.1.2 Dimostrazione Halting Problem

Andiamo ora a vedere come puo' essere dimostrato che l'halting problem e' indecidibile.
Per prima cosa dobbiamo definire la controparte concreta delle frecce e punti dello schema:

- I punti *COSE* sono insiemi di tutti i programmi o dati possibili. Sfruttando la **Numerazione di Godel** possiamo attribuire a ognuno di questi un numero naturale. Quindi *COSE* diventa $\mathbb{N}$.

- I punti *VALORI* sono tutti i possibili output di un programma, quindi dei dati. Quindi *VALORI* diventa $\mathbb{N}$.
- La freccia X diventa il programma *HALT*, ovvero la funzione totale che prende un *Programma* P e un *Dato* D (entrambe $\in \mathbb{N}$) tale che:

$$X(P, D) = \begin{cases} \text{"termina"} & P \text{ termina su } D \\ \text{"diverge"} & P \text{ non termina su } D \end{cases}$$

  Per ora assumiamo l'esistenza di questo programma.
- La freccia D e' il programma che implementa la funzione totale $\mathbb{N} \to \mathbb{N} \times \mathbb{N}$

$$D(x) = (x, x)$$

  che e' facile da costruire e quindi esiste.

$$\begin{tikzcd}
\mathbb{N} \times \mathbb{N} \arrow[r, "X"] & \mathbb{N} \arrow[d, "I"] \\
\mathbb{N} \arrow[u, "D"] \arrow[r, "S"'] & \mathbb{N}
\end{tikzcd}$$

Non e' chiaro come dobbiamo definire I in modo da dimostrare l'assurdo, quindi lavoriamo top-down.
Dato che una composizione di programmi e' un programma, anche S e' sicuramente un programma, quindi una funzione parziale che puo' terminare o divergere dato un input.
Essendo S un programma, e' possibile rappresentarlo con un numero $k \in \mathbb{N}$. Analizziamo $S(k)$:

- $S(k)$ termina: in questo caso $X(k, k) =$ "termina", quindi $S(k) = I($"termina"$)$. Per rimanere coerente I deve terminare sull'input "termina".
- $S(k)$ non termina: in questo caso $X(k, k) =$ "diverge", quindi $S(k) = I("diverge")$. Per rimanere coerente I deve divergere sull'input "diverge".

Ma noi non vogliamo la coerenza, vogliamo costruire S in modo da dimostrare l'assurdo. Quindi definiamo I come la funzione parziale che:

- Diverge sull'input "termina"
- Termina sull'input "diverge"

Abbiamo quindi che:

- Se $X(k, k) =$ "termina", allora $S(k)$ diverge
- Se $X(k, k) = "diverge"$, allora $S(k)$ termina

Queste due implicazioni sono in opposizione alla definizione del programma *HALT*, che quindi non puo' esistere.

1.2 Tipi di problemi

Ci sono due categorie:

- **Ricerca**: vogliamo calcolare un risultato arbitrario, ovvero *ricercano* un output vario.
- **Decisione**: l'output e' *booleano*.

Queste due tipologie non sono completamente sganciate, esiste una relazione.
Dato un problema di ricerca, possiamo ricavare la versione "decisionale" che e' piu' facile da calcolare.

Example 1.2.1 (Somma)
Ricerca: dati a, b calcolare la somma $c = a + b$.
Decisione: dati a, b, c determinare se $c = a + b$ (vero o falso).

Per questo motivo parleremo quasi totalmente solo di problemi di decisione.

Note:
E' da qui che deriva il termine "decidibile".

1.2.1 Problemi di decisione

Theorem 1.2.1
Tutti i problemi di decisione possono essere ridotti al riconoscimento di uno specifico linguaggio.

Example 1.2.2
PATH $< G, s, t >$: vogliamo stabilire se esiste un percorso da s a t.
Mostriamo di poter trasformare questo problema decisionale in un problema che chiede se una "parola" appartiene a uno specifico linguaggio.

$$L_{\text{PATH}} = \{< G, s, t > | G \text{ e' un Grafo } \wedge s, t \text{ sono nodi del grafo } G \wedge \exists \text{ un percorso da } s \rightarrow t\}$$

Ci riduciamo quindi a decidere se una data tripla appartiene a L_{PATH} o meno.

Quindi, mentre studiamo decidibilita' dei problemi guarderemo linguaggi e i loro *automi*.

1.3 Linguaggi

1.3.1 Definizioni

Definition 1.3.1: Alfabeto

$\Sigma = \{a, b, c, ...\}$ e' l'*alfabeto*
E' un insieme di simboli.

Definition 1.3.2: Parola

w e' una parola o striga su Σ e' un concatenamento di 0 o piu' simboli di Σ.

Definition 1.3.3: Kleene

def

Definition 1.3.4: Linguaggio

Sottoinsieme delle parole che si possono costruire su un determinato alfabeto Σ:

$$L \subseteq \Sigma^*$$

Definition 1.3.5

Dato un linguaggio L **fissato** per una stringa w, decidere se:

$$w \in L$$

Il problema di appartenenza a un linguaggio L (**fissato!**) e' definito come:
dato il linguaggio L, prendere in input una parola w e decidere se $w \in L$.
input: stringa output: booleano
Perche' vogliamo usare automi? : e' un formalismo piu' basico e piu' semplice da studiare rispetto a un arbitrario linguaggio di prorammazione.

1.3.2 Automi

Vabbe' introduce gli automi, il Guerriero ci ha gia preparato adeguatamente.
Alright whatsapp.

1.3.3 Macchia di Turing

Automa piu' potente che Turing si invento per studiare i problemi importanti da chiudere, fra cui l'automatizzazione dei teoremi.
E' nato per formalizzare il calcolo automatico in un tempo in cui non esistevano i computer, era la versione "automatizzata" di lui che scriveva su carta.
Iniziamo con una spiegazione informale:
E' una macchina caratterizzata da un nastro con delle cellette
In ognuna cella c'e' un simbolo, ed e' infinito in entrambe le direzioni
Ha una testina posizionata su una cella da cui puo' leggere il simbolo
La testina puo' spostarsi a dx o sx, e a differenza degli automi visti puo' benissio tornare indietro e cambiare la decisione presa prima (backtracking)
La testina puo' anche scrivere su una cella, perdendo cio' che c'era prima (una seconda differenza)
Per noi sta roba e' un algoritmo, dato che la macchina di Turing, in quanto automa, ha vari stati di funzionamento

- Legge simbolo cella
- In base allo stato, sovrascrive qualcosa
- Sposto la testina avanti o indietro
- Cambio lo stato

Example 1.3.1

Progetta una MdT che riconosca il linguaggio

$$L = \{a^m b^m | m \geqslant 0\}$$

La stringa da decidere si trova sul nastro da sx a dx e la testina si trova sopra il primo simbolo.
Un algoritmo e' un filmato di quello che succede nella nostra testa per risolvere un problema.

- Leggi cella, se e' vuota fine (ok)
- altrimenti se e' b fine (no)
- se e' a, cancella
- spostati a dx finche' la cella e' bianca
- vai a sx di uno e leggi
- se e' a, fine (no)
- se e' b, cancella e vai a sx finche e' bianca
- muoviti di uno a dx

TODO: disegna automa

Definizioni formali

Definition 1.3.6: Macchina di Turing

E' la settupla

$$M = \{Q, q_0, F, \Sigma, \Gamma, b/, \delta\}$$

Dove:

- Q e' l'insieme di stati
- q_0 e' lo stato iniziale
- F e' l'insieme degli stati finali
- Σ e' l'insime dei simboli in input
- Γ e' l'insieme dei simboli che la macchina puo' scrivere $\Sigma \subset \Gamma$
- $b/$ e' il simbolo di cella vuota ("blank") $b/\in \Gamma$
- δ e' la funzione

$$Q \times \Gamma \to Q \times \Gamma \times \{\leftarrow, \rightarrow\}$$

La macchina di turing ha stati finiti (come gli altri automi) e il suo programma e' fissato (non cambia). Quindi come fa a eseguire qualunque computazione? Vedremo.
δ codifica le funzioni di transizioni (gli archi dell'automa)

Definition 1.3.7: Configurazione

Sia $M = \{Q, q_0, F, \Sigma, \Gamma, \not{b}, \delta\}$ una MdT. Si definisce *configurazione* una stringa $\alpha q \beta$ dove

- $q \in Q$ è lo stato corrente della macchina.
- $\alpha, \beta \in \Gamma^*$ sono le stringhe di simboli che si trovano a sinistra e a destra della testina.

Note:

Si può esprimere informalmente come una "fotografia" dello stato corrente di computazione della macchina:

- Stato macchina
- Posizione testina
- Cosa c'e' sul nastro

Example 1.3.2

$$aq_1abb$$

dove q_1 e' lo stato della macchina e la testina legge il carattere a dx dello stato.

$$aabq_3b/$$

sta leggendo la prima cella bianca a dx
Nota: i caratteri bianchi ci permettono di precisare la posizione della testina, ma non ne dobbiamo mettere infiniti obv

Partendo dalla configurazione, usando la funzione di transizione siamo in grado di calcolare il passo che eseguira'

Definition 1.3.8: Configurazione Accettante

Quando al suo interno si trova uno stato accettante

Definition 1.3.9: Legal Successor

Data una MdT M e sia c_1 una sua configurazione, c_2 si dice *legal successor* di c_1 rispetto a M se c_2 si ottiene da c_1 in esattamente un passo di computazione mediante l'applicazione di una transizione definita nella funzione di transizione δ.
E lo si denota con:

$$c_1 \vdash_M c_2$$

Example 1.3.3 (Legal successor)
Sia M una MdT e sia $c_1 = aq_1b$ una sua configurazione. Se $\delta(q_1, b) = (q_2, a, \rightarrow)$, allora $c_2 = aaq_2b$ e' legal successor di c_1 rispetto a M

Definition 1.3.10: Configurazione iniziale

si definisce configurazione iniziale di una MdT M e di una stringa $w \in \Sigma^*$ la configurazione $c_1 = q_0w$

Definition 1.3.11: Configurazione Finale

Configurazione che non ammette legal successor secondo M

Definition 1.3.12: Computazione Parziale

Data una MdT M, una *computazione parziale* per M e' una sequenza:

$$c_1, c_2, \ldots, c_n$$

dove

$$c_1 \vdash_M c_2 \vdash_M \cdots \vdash_M c_n$$

Note:
Non è detto che c_1 sia il punto di partenza e c_n sia l'ultima configurazione della computazione, per questo è parziale.

Definition 1.3.13: Computazione

Data una MdT M e una stringa $w \in \Sigma^*$, una *computazione* per M e' una computazione parziale $c_1, c_2, \ldots, c_n$ per M dove c_1 è una configurazione iniziale e c_n è una configurazione finale

Possiamo ora definire quando una MdT accetta un input

Definition 1.3.14: Accettazione

Una MdT accetta w se si arresta su uno stato accettante, e rifiuta w se non si arresta su uno stato accettante.

Note:
Da questa definizione, se una MdT non si arresta allora l'input non e' accettato. Quindi per non accettare puo' anche divergere. Questa roba qua per noi e' un po' cringe.

Definition 1.3.15: Linguaggio di una MdT

Linguaggio

$$\mathcal{L}(M) = \{w | M(w) = 1\}$$

Definition 1.3.16: Decide

Sia L un linguaggio, M *decide* L sse $\forall w \in \Sigma^*$

$$w \in L \implies M \text{ accetta } w$$

$$w \notin L \implies M \textit{ si arresta } \text{e dice di "no" } w$$

Definition 1.3.17: Riconosce

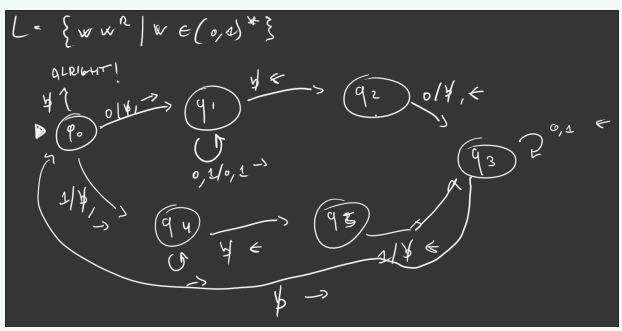
Sia L un linguaggio, M *riconosce* L sse $\forall w \in \Sigma^*$

$$w \in L \implies M \text{ accetta } w$$

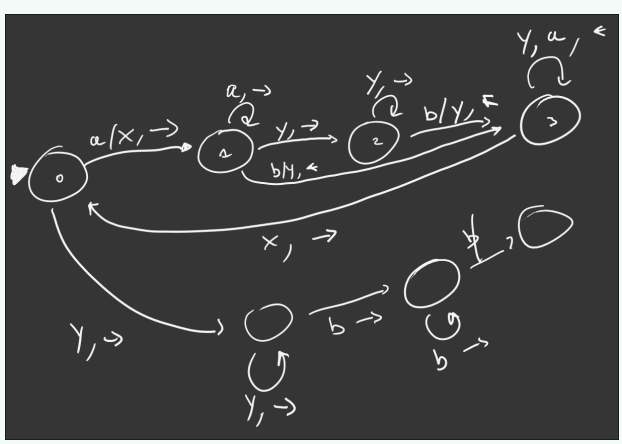
$$w \notin L \implies M \text{ non accetta } w$$

Esercizi

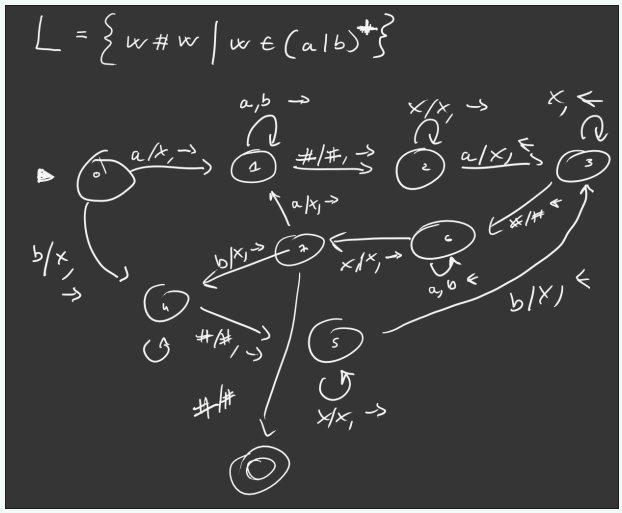
Example 1.3.4



Example 1.3.5



Example 1.3.6



Chapter 2

Macchine di Turing e Linguaggi

2.1 Linguaggi

2.1.1 Definizioni

Definition 2.1.1: Alfabeto

$\Sigma = \{a, b, c, ...\}$ e' l'*alfabeto*
E' un insieme di simboli.

Definition 2.1.2: Parola

w e' una parola o striga su Σ e' un concatenamento di 0 o piu' simboli di Σ.

Definition 2.1.3: Kleene

def

Definition 2.1.4: Linguaggio

Sottoinsieme delle parole che si possono costruire su un determinato alfabeto Σ:

$$L \subseteq \Sigma^*$$

Definition 2.1.5

Dato un linguaggio L **fissato** per una stringa w, decidere se:

$$w \in L$$

Il problema di appartenenza a un linguaggio L (**fissato!**) e' definito come:
dato il linguaggio L, prendere in input una parola w e decidere se $w \in L$.
input: stringa output: booleano
Perche' vogliamo usare automi? : e' un formalismo piu' basico e piu' semplice da studiare rispetto a un arbitrario linguaggio di prorammazione.

2.1.2 Automi

Vabbe' introduce gli automi, il Guerriero ci ha gia preparato adeguatamente.
Alright whatsapp.

2.2 Macchia di Turing

Automa piu' potente che Turing si invento per studiare i problemi importanti da chiudere, fra cui l'automatizzazione dei teoremi.
E' nato per formalizzare il calcolo automatico in un tempo in cui non esistevano i computer, era la versione "automatizzata" di lui che scriveva su carta.
Iniziamo con una spiegazione informale:
E' una macchina caratterizzata da un nastro con delle cellette
In ognuna cella c'e' un simbolo, ed e' infinito in entrambe le direzioni
Ha una testina posizionata su una cella da cui puo' leggere il simbolo
La testina puo' spostarsi a dx o sx, e a differenza degli automi visti puo' benissio tornare indietro e cambiare la decisione presa prima (backtracking)
La testina puo' anche scrivere su una cella, perdendo cio' che c'era prima (una seconda differenza)
Per noi sta roba e' un algoritmo, dato che la macchina di Turing, in quanto automa, ha vari stati di funzionamento

- Legge simbolo cella
- In base allo stato, sovrascrive qualcosa
- Sposto la testina avanti o indietro
- Cambio lo stato

Example 2.2.1

Progetta una MdT che riconosca il linguaggio

$$L = \{a^m b^m | m \geqslant 0\}$$

La stringa da decidere si trova sul nastro da sx a dx e la testina si trova sopra il primo simbolo.
Un algoritmo e' un filmato di quello che succede nella nostra testa per risolvere un problema.

- Leggi cella, se e' vuota fine (ok)
- altrimenti se e' b fine (no)
- se e' a, cancella
- spostati a dx finche' la cella e' bianca
- vai a sx di uno e leggi
- se e' a, fine (no)
- se e' b, cancella e vai a sx finche e' bianca
- muoviti di uno a dx

TODO: disegna automa

2.2.1 Definizioni formali

Definition 2.2.1: Macchina di Turing

E' la settupla

$$M = \{Q, q_0, F, \Sigma, \Gamma, b/, \delta\}$$

Dove:

- Q e' l'insieme di stati
- q_0 e' lo stato iniziale
- F e' l'insieme degli stati finali
- Σ e' l'insime dei simboli in input
- Γ e' l'insieme dei simboli che la macchina puo' scrivere $\Sigma \subset \Gamma$
- $b/$ e' il simbolo di cella vuota ("blank") $b/\in \Gamma$
- δ e' la funzione

$$Q \times \Gamma \to Q \times \Gamma \times \{\leftarrow, \rightarrow\}$$

La macchina di turing ha stati finiti (come gli altri automi) e il suo programma e' fissato (non cambia). Quindi come fa a eseguire qualunque computazione? Vedremo.
δ codifica le funzioni di transizioni (gli archi dell'automa)

Definition 2.2.2: Configurazione

E' una "fotografia" dello stato corrente di computazione della macchina:

- Stato macchina
- Posizione testina
- Cosa c'e' sul nastro

Example 2.2.2

$$aq_1abb$$

dove q_1 e' lo stato della macchina e la testina legge il carattere a dx dello stato.

$$aabq_3b/$$

sta leggendo la prima cella bianca a dx
Nota: i caratteri bianchi ci permettono di precisare la posizione della testina, ma non ne dobbiamo mettere infiniti obv

Partendo dalla configurazione, usando la funzione di transizione siamo in grado di calcolare il passo che eseguira'

Definition 2.2.3: Configurazione Accettante

Quando al suo interno si trova uno stato accettante

Definition 2.2.4: Legal Successor

Data una MdT M e sia c_1 una sua configurazione, c_2 si dice *legal successor* di c_1 sse:

$$c_1 \vdash_M c_2$$

Definition 2.2.5: Configurazione Finale

Configurazione che non ha legal successor

Definition 2.2.6: Computazione Parziale

Data una MdT M, una *computazione parziale* per M e' una sequenza:

$$c_1, c_2, ...c_n$$

dove

$$c_1 \vdash_M c_2 \vdash_M ... \vdash_M c_n$$

Note:

Non e' detto che c_1 sia il punto di partenza e c_n sia l'ultima configurazione della configurazione, per questo e' parziale.

Definition 2.2.7: Computazione

Data una MdT M e una stringa $w = w_1 w_2 ... w_l$, una *computazione* per M e' una computazione parziale per M dove

$$c_1 = q_0 w_1 w_2 ... w_l$$

e c_n e' una configurazione finale

Possiamo ora definire quando una MdT accetta un input

Definition 2.2.8: Accettazione

Una MdT accetta w se si arresta su uno stato accettante, e rifuita w se non e' vero che si arresti in uno stato accettante.

Note:

Da questa definizione, se una MdT non si arresta allora l'input non e' accettato. Quindi per non accettare puo' anche divergere. Questa roba qua per noi e' un po' cringe.

Definition 2.2.9: Linguaggio di una MdT

Linguaggio

$$\mathcal{L}(M) = \{w | M(w) = 1\}$$

Definition 2.2.10: Decide

Sia L un linguaggio, M *decide* L se $\forall w \in \Sigma^*$

$$w \in L \implies M \text{ accetta } w$$

$$w \notin L \implies M \textit{ si arresta} \text{ e dice di "no" } w$$

Definition 2.2.11: Riconosce

Sia L un linguaggio, M *riconosce* L se $\forall w \in \Sigma^*$

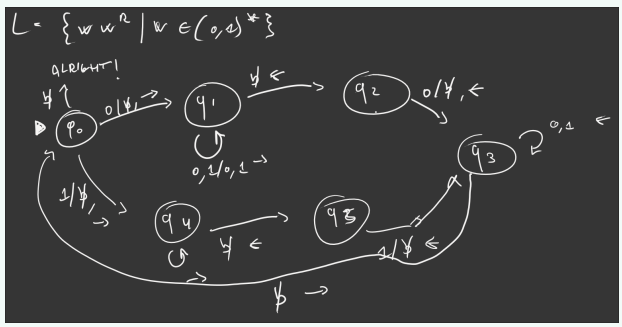
$$w \in L \implies M \text{ accetta } w$$

$$w \notin L \implies M \text{ non accetta } w$$

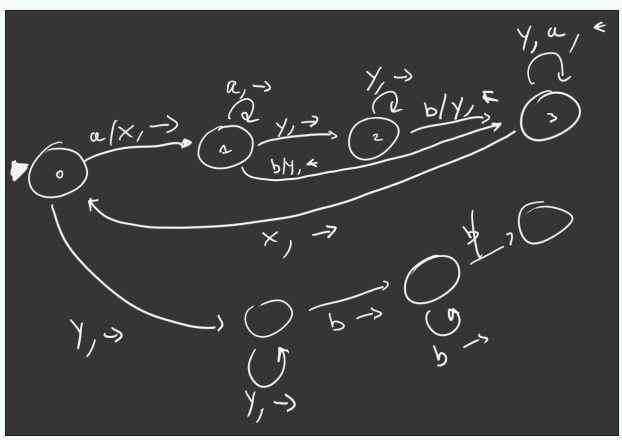
2.2.2 Esercizi

TODO: sarebbe figo fare gli automi in latex, se ho sbatti ci provo. Ah e alcune soluzioni sono imprecise le dovrei fixare.

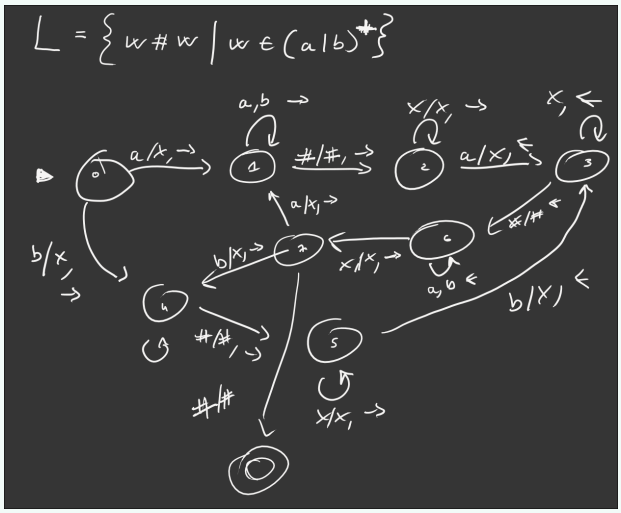
Example 2.2.3



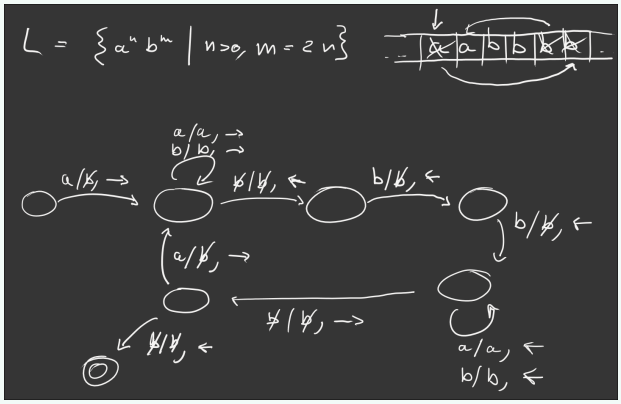
Example 2.2.4



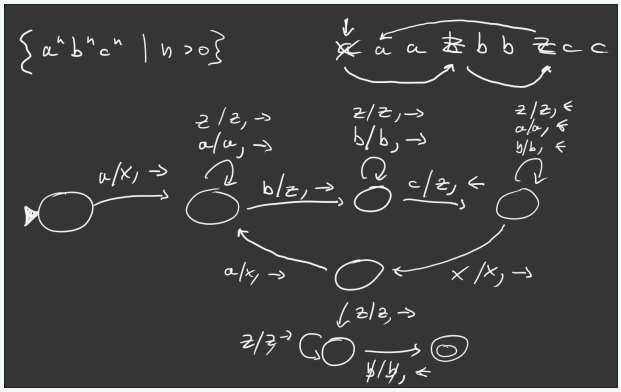
Example 2.2.5



Example 2.2.6



Example 2.2.7



2.3

Le macchine che abbiamo visto finora sono molto tediose.
Per esempio, per riconoscere il linguaggio $L = \{ww^R | w \in (0|1)^*\}$ e' stato necessario costruire due rami (uno per lo 0 e uno per l'1). Sarebbe comodo poter usare un solo ramo che in qualche modo codifica entrambe i rami,

vediamo come possiamo fare:

Definition 2.3.1: Memoria nello stato

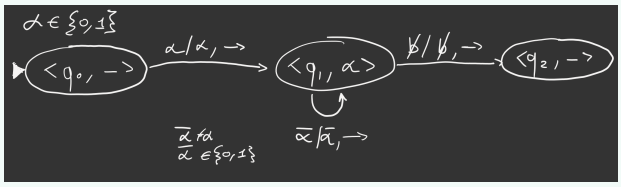
Data una MdT, si dice che questa ha **memoria** (nello stato) se ad ogni stato e' associata una memoria di uno o piu' simboli:

$$\langle q_1, -, ..., - \rangle$$

Il numero di simboli in memoria e' fissato, ovvero non puo' cambiare durante l'esecuzione.

Example 2.3.1

Vogliamo riconoscere $L = 10^*|01^*$



Notiamo che la macchina con memoria sopra non e' altro che una rappresentazione piu' compatta di una normale MdT con due rami al posto di uno.

Proposition 2.3.1 Equipotenza macchine con memoria

Data una MdT con memoria nello stato, e' sempre possibile costruire una MdT senza memoria che riconosce lo stesso linguaggio.

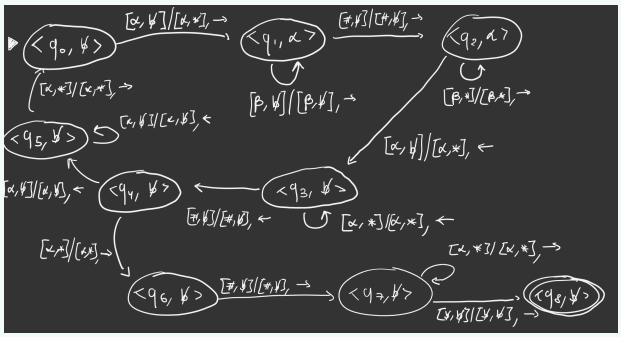
Dimostrazione: Boh in verita' non dimostra sto risultato, dice solo che alla fine le macchine con memoria sono un prodotto cartesiano di stati con simboli.

Definition 2.3.2: Macchine Multitraccia

Multiple tracce lette in sincrono, la testina si muove tutta insieme ma possiamo leggere e scrivere singolarmente sulle tracce.

Example 2.3.2

Vogliamo riconoscere $L = \{w\#w | w \in (a|b)^+\}$:



Proposition 2.3.2 Equipotenza macchine multitraccia

Data una macchina multitraccia, e' sempre possibile costruire una MdT classica che riconosce lo stesso linguaggio.

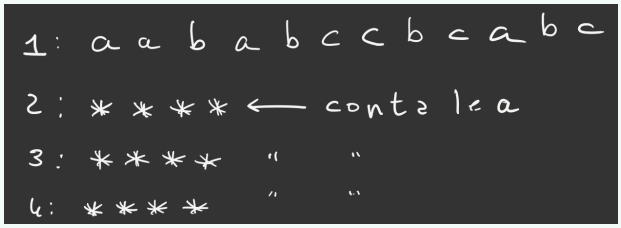
Dimostrazione: Data una macchina multitraccia di n tracce con alfabeti Γ_i per ogni traccia i, possiamo costruire una MdT con $\Gamma = \Gamma_1 \times ... \times \Gamma_n$ che si comporta esattamente come la macchina multitraccia.

Definition 2.3.3: Macchine Multinastro

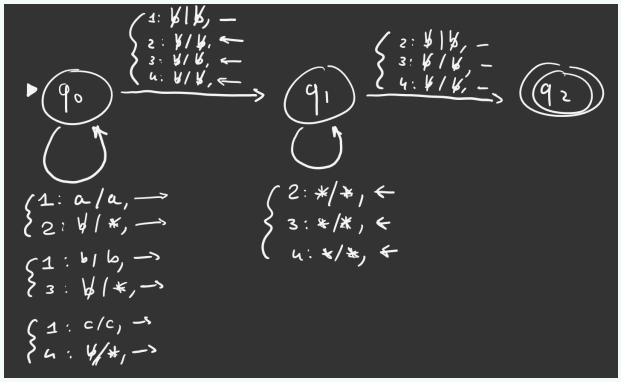
MdT che ha piu' nastri con testine indipendenti. Il numero dei nastri e' fissato a costruzione. E' possibile che una testina rimanga ferma.

Example 2.3.3

Costruiamo la macchina multinastro che riconosce $L = \{w | w \in (a|b|c)^* \wedge$ il numero delle a, b e c sono uguali$\}$. Vediamo l'idea:



Ora costruiamo l'automa:



Proposition 2.3.3 Equipotenza di macchine multinastro

Data una macchina multinastro, e' sempre possibile costruire una MdT classica che riconosce lo stesso linguaggio.

Dimostrazione: Data una macchina multinastro e' possibile costruire una multitraccia equivalente con $2n$ tracce (dove n e' il numero dei nastri) semplicemente usando le tracce dispari come "puntatori" che indicano l'ipotetica posizione della testina, come se fosse multinastro.
Ad ogni passo, collochiamo inizialmente la testina sul primo puntatore a sx per poi scansionare tutto fino all'ultimo puntatore. I simboli sotto ogni puntatore vengono salvati nella memoria, e la testina ritorna indietro cambiando in modo adeguato i simboli puntati e la posizione dei puntatori.

E' il modello di calcolo piu' flessibile che abbiamo a disposizione. Ma abbiamo visto che per simulare il suo comportamento con una macchina multitraccia impieghiamo piu' transizioni, ma quante di piu'?

Dimostrazione: Analizziamo il caso peggiore dopo m passi. Questo avviene quando una testina della macchina si sposta a sx ad ogni passo e una si sposta a dx ad ogni passo, in modo che distino $2m$ celle.
Quindi una macchina multinastro avra' bisogno di percorrere due volte la distanza (passaggio in avanti e indietro) e deve poi aggiornare ogni puntatore (uno per ogni nastro), che nel caso peggiore comporta altre due transizioni. Alla fine abbiamo che un solo passo di una macchina multinastro (k nastri) richiede nel caso peggiore $4m + 2k$ passi di una macchina multitraccia.

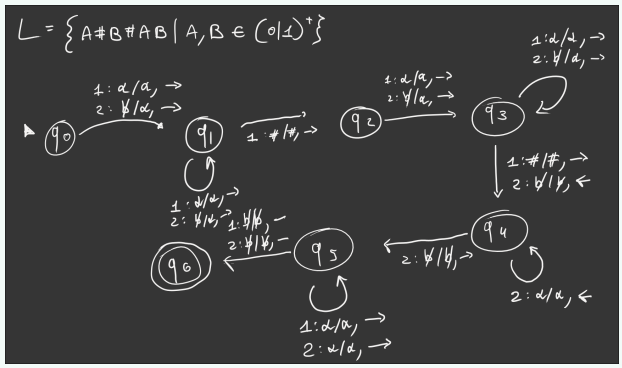
Possiamo quindi dire che una computazione che richiede n passi ad una macchina multinastro richiedera'

$$\sum_{i=1}^{n} 4i + 2k \leqslant \sum_{i=1}^{n} 4n + 2k \leqslant n(4n + 2k)$$

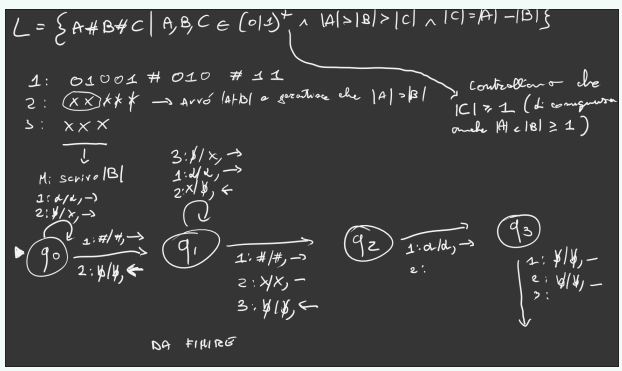
Quindi e' $O(n^2)$.

2.3.1 Esercizi

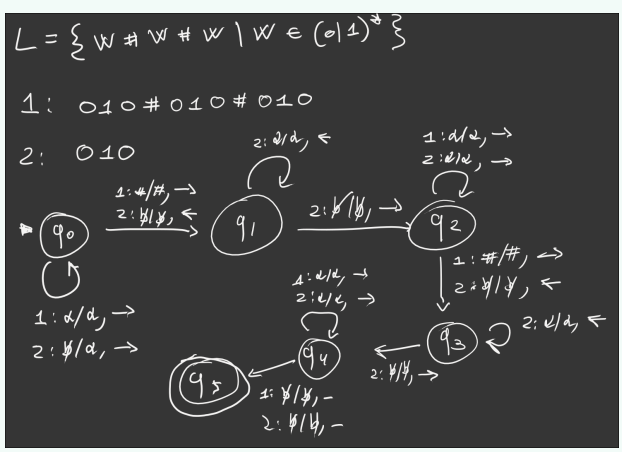
Example 2.3.4



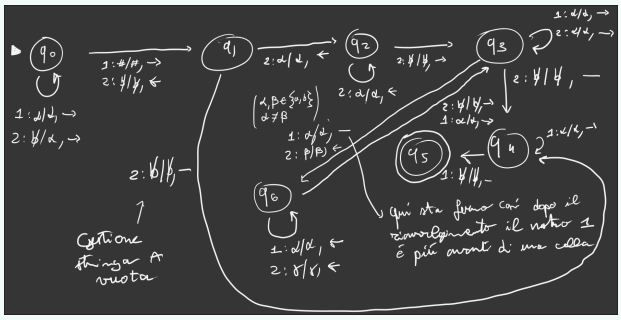
Example 2.3.5



Example 2.3.6



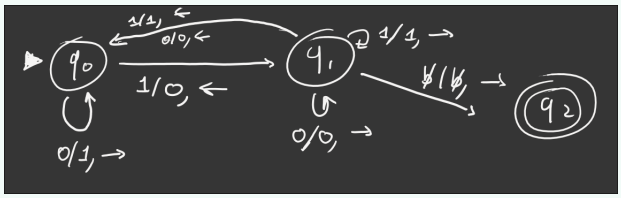
Example 2.3.7



2.3.2 Macchine Non Deterministiche

In verita' adesso introduciamo un altro tipo di macchina crazy che ci servira', vediamo un esempio:

Example 2.3.8



Notiamo che e' una macchina particolare perche' la sua funzione di transizione specifica un insieme di possibili passi data una configurazione.

Definition 2.3.4: Macchina Non-deterministica

N e' una macchina non-deterministica se e':

$$N = \langle \Sigma, \Gamma, b\, Q, q_0, F, \delta \rangle$$

dove δ cambia rispetto alla classica macchina:

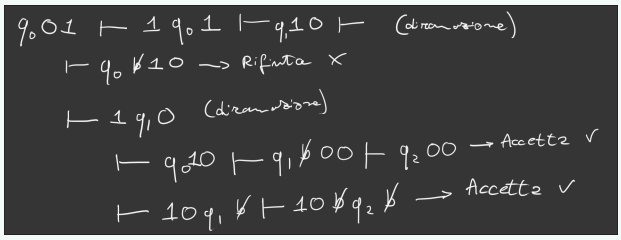
$$\delta : Q \times \Gamma \to 2^{Q \times \Gamma \times \{\leftarrow, \rightarrow\}}$$

Note:

Le macchine deterministiche sono un sottoinsieme degenere delle macchine non deterministiche dove l'immagine e' formata solo da insiemi di cardinalita' 1.

Example 2.3.9

Supponiamo di voler vedere come funziona la macchina di prima su un input 01:



Ma accetta o rifuita?

Supponiamo di avere una bella MdT non-deterministica, cosa darebbe in output? Questa e' una domanda a trabocchetto perche' in verita' non potrebbe esistere (le macchine quantistiche sono meno potenti). Come vogliamo definire se una stringa e' accettata quindi?
Ci possiamo studiare l'*albero di computazione*, che usa le configurazioni (che hanno significato analogo a quelle deterministiche) con le stesse definizioni di stato iniziale, finale e accettante, pero' cambiano i legal succesors.
Quindi i nodi sono tutte le configurazioni che una macchina non deterministica puo' provare processando la stringa w. La radice e' la config iniziale, e abbiamo un arco fra configurazioni $a \to b$ se la config b e' legal successor della config a. Quindi la computazione e' un albero e non una lista.
La stringa w in input viene accettata quando nell'albero di computazione esiste almeno un percorso dalla radice a uno stato accettante.
Ma sta roba e' capace di riconoscere linguaggi che le macchine deterministiche non sono in grado di descrivere?

Proposition 2.3.4

Una macchina di turing non deterministica ha lo stesso potere espressivo di quelle deterministiche.

: La macchina non determinista N puo' essere ricondotta a una determinista M

$$N \longrightarrow M \ \mathcal{L}(N) = \mathcal{L}(M)$$

Immaginiamo che N sia mononastro e M binastro.
Stiamo esplorando l'albero in BFS dato che, se ci fosse una computazione infinita nell'albero e lo esploriamo in DFS, e' possibile che la macchina deterministica si "blocchi" a esplorare il ramo infinito quando forse puo' esserci un'altro ramo che in realta' e' accettante.

Ma questa simulazione quanto costa?
Supponiamo che il numero massimo di figli nell'albero di computazione abbia al massimo c figli, allora in numero di nodi totali di un albero di altezza m e'

$$O(e^m)$$

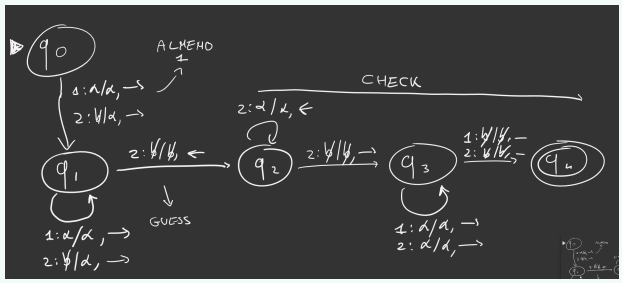
Quindi anche il costo per simulare la BFS di questo albero sara' esponenziale. Ma in realta' non siamo sicuri che questo sia il limite, e non siamo neanche stati in grado di dimostrare che questa cosa non e' possibile. P vs NP e' incentrata proprio su questa questione
Ipotesi di Church-Turing:
Tutto cio' che e' calcolabile e' calcolabile da una macchina di Turing
E' self referential, non abbiamo una definizione di calcolabile, semplicemente non abbiamo sviluppato altri formalismi di calcolo con potere espressivo piu' alto. Quindi appunto non e' un teorema
Introduciamo le prime due classi di calcolabilita':

- **RE**: ci sono tutti i linguaggi semi-decidibili, ovvero per i quali esiste una MdT che accetta (non decide) il linguaggio (potrebbe non terminare).
- **R**: inclusa in RE, classe dei problemi di decisione ricorsivi (linguaggi ricorsivi), ovvero per i quali esiste una MdT che quindi da' garanzia di risposta. Sono quindi linguaggi decidibili.

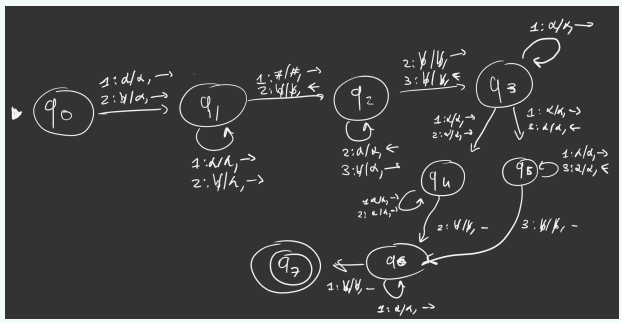
Definition 2.3.5: Classe

Semplicemente un insieme di linguaggi.

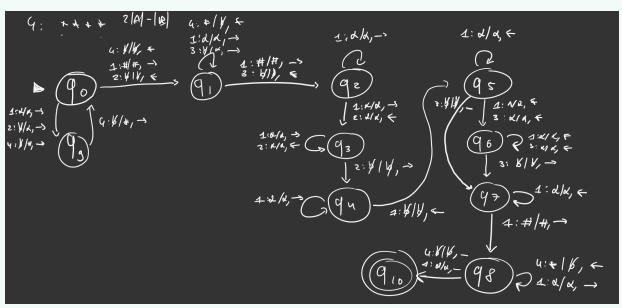
Example 2.3.10



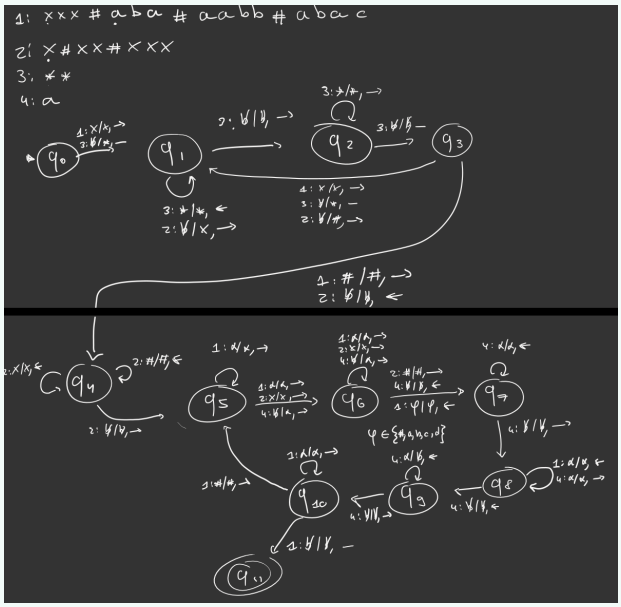
Example 2.3.11



Example 2.3.12



Example 2.3.13



Ha spammato degli esercizi, onesto possiamo anche guardarli da virtuale.
Abbiamo introdotto le macchine nondeterministiche per affrontare i problemi in modo piu' semplice, ora che siamo forti useremo algoritmi nondeterministici per risolvere problemi in modo da dimostrare la loro decidibilita'.

2.4 Macchine di turing universali

Le macchine di turing presentate finora sono "statiche" ovvero accettano/decidono un solo linguaggio. Sono programmate solo per fare una cosa, dato che la funzione di transizione è fissata.
Tuttavia i computer che usiamo tutti i giorni sono "dinamici" ovvero possono eseguire qualunque programma, ovvero la funzione di transizione è più "ricca" di informazioni, dato che deve codificare anche il programma che deve eseguire.
Quindi ci poniamo il problema: è possibile costruire una macchina di turing che possa eseguire qualunque programma? La risposta è sì, ed è la macchina di turing universale.

2.4.1 Codifica della macchina di turing

Consideriamo una MdT il cui alfabeto di input abbia solo due simboli $\Sigma = \{0, 1\}$, mentre l'alfabeto di nastro può essere $\Gamma = \{0, 1, \not{b}, \dots\}$.
La funzione di transizione ha generalmente la forma:

$$\delta(Q_i, X_j) = (Q_k, X_l, D_m)$$

Example 2.4.1

$$\delta(q_3, \not{b}) = (q_5, 0, \rightarrow)$$

Codifichiamo i simboli Q_i, X_j, Q_k, X_l, D_m partendo dai loro pedici:

$$Q_i = 0^i, \quad X_j = 0^j, \quad \text{sx} = D_1, \quad \text{dx} = D_2$$

Note:

Per convenzione fissiamo:

- $x_1 = 0,\ x_2 = 1,\ x_3 = \not{b},\ x_4, \dots$
- q_1 è sempre lo stato iniziale

- q_2 è sempre il singolo stato accettante

Ogni singola transizione c_i viene quindi codificata come una stringa binaria in cui i vari campi sono separati da 1. Ad esempio, la transizione $\delta(q_3, \not{b}) = (q_5, x_1, D_2)$ viene codificata come:

$$c_1 = \underbrace{000}_{q_3}\ 1\ \underbrace{000}_{X_3}\ 1\underbrace{00000}_{q_5}1\ \underbrace{0}_{X_1}\ 1\ \underbrace{00}_{D_2}$$

Per unire più transizioni in un'unica stringa, le istanze c_i vengono separate da 11:

$$c_1 11 c_2 11 c_3 11 c_4 \cdots$$

Note:

Esistono infiniti modi per codificare una MdT, ma scegliamo questo. L'obiettivo è poter comunicare a una MdT come comportarsi rispetto a un'altra, quindi ogni stringa binaria può essere messa in associazione con una possibile MdT.
Ci saranno inevitabilmente stringhe binarie che non codificano nessuna MdT valida (perché non rispettano il formato). Per convenzione, assumiamo che tali stringhe corrispondano a una macchina con un unico stato non accettante.

2.4.2 Definizione formale

Definition 2.4.1: Macchina di Trueing universale

La Macchina di Turing Universale M_u e' una macchina che riceve in input una stringa binaria che codifica una macchina M e poi una stringa w

$$(M, w)$$

La macchina M_u e' in grado di dirci se M avrebbe accettato o meno w

Definition 2.4.2: Linguaggio universale

Linugaggio che accetta una macchina universale M_u

$$L_u = \{\langle M, w\rangle | M \models w\}$$

2.4.3 Processo di simulazione

La macchina M_u è una macchina a **quattro nastri**:

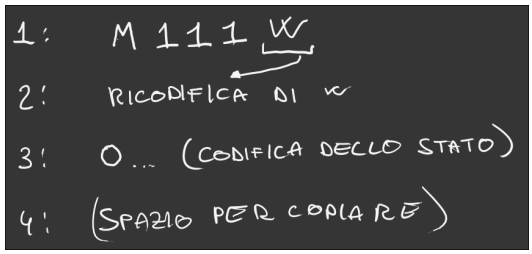
- **Nastro 1 (input)**: contiene la coppia M `111` w, dove M e w sono separati da tre 1 consecutivi. Questo separatore è non ambiguo poiché nella codifica di M non compaiono mai tre 1 di fila.
- **Nastro 2 (nastro simulato)**: contiene la ricodifica di w secondo i simboli simulati.

  $$C(w_i) = \begin{cases} 0 & w_i = 0 \\ 00 & w_i = 1 \end{cases}$$

  e la codifica completa

  $$C(w_1)1C(w_2)1...C(w_n)$$

- **Nastro 3 (stato simulato)**: tiene traccia dello stato corrente di M durante la simulazione.
- **Nastro 4 (lavoro)**: nastro ausiliario per operazioni di copia e spostamento.



Se la codifica in input non e' sensata, allora la macchina universale simula la macchina che rifiuta tutto. Altrimenti la simulazione procede nelle seguenti fasi:

1. **Inizializzazione**: M_u ricodifica w sul nastro 2 secondo lo schema dei simboli simulati. Ad esempio:

$$w = \underbrace{0}_{X_1}\ \underbrace{1}_{X_2}\ \underbrace{0}_{X_1} \implies \texttt{1 0 1 0 1}$$

   Scrive sul nastro 3 lo stato iniziale $q_1 = 0$.

2. **Ciclo di simulazione**: ad ogni passo, M_u conosce lo stato corrente di M (nastro 3) e il simbolo correntemente letto (nastro 2). Cerca quindi sul nastro 1 la transizione corrispondente $\delta(q_i, x_j)$ e:
   - aggiorna il nastro 3 con il nuovo stato q_k
   - sovrascrive il simbolo sul nastro 2 con x_l
   - sposta la testina sul nastro 2 nella direzione D_m
   - usa il nastro 4 come area di lavoro se è necessario ridimensionare il nastro 2

3. **Terminazione**: la simulazione termina quando M raggiunge una configurazione finale:

$$M \text{ accetta } w \implies M_u \text{ accetta}$$
$$M \text{ rifiuta } w \implies M_u \text{ rifiuta}$$
$$M \text{ non termina su } w \implies M_u \text{ non termina}$$

Note:

I nostri computer sono esattamente delle M_u. La loro funzione di transizione universale è fissata nell'hardware: è il **microcodice della CPU**. Questo risponde alla domanda su come mai le MdT classiche non si possono "programmare" mentre i computer sì.

2.5 Problemi Ricorsivi e Ricorsivamente Enumerabili

Abbiamo introdotto le classi di calcolabilità R e RE, e abbiamo detto che i problemi possono stare in R e in RE. Ma possono esistere problemi che stanno fuori da RE? Sì, e ora li andremo a vedere.

2.5.1 Enumerare stringhe e macchine

Riconsideriamo macchine che processano in input stringhe binarie. Se prendiamo una stringa binaria e ci mettiamo un 1 davanti, possiamo associarle un numero intero:

$$1 \circ 00010\ldots$$

In questo modo possiamo enumerare tutte le stringhe binarie:

$$\underbrace{1 \circ \varepsilon}_{1}\quad \underbrace{1 \circ 0}_{2}\quad \underbrace{1 \circ 1}_{3}\quad \underbrace{1 \circ 01}_{4}\quad \underbrace{1 \circ 10}_{5}\quad \cdots$$

Le stringhe con indice più basso sono tendenzialmente più corte; stringhe della stessa lunghezza sono ordinate lessicograficamente. Le stringhe binarie sono **infinite numerabili**.
Lo stesso ragionamento vale per le MdT: grazie alla codifica binaria introdotta nella sezione precedente, possiamo mettere un 1 davanti alla codifica di ogni MdT e ottenere un numero intero. Quindi **possiamo contare le macchine di Turing**, chiamandole $M_1, M_2, M_3, \ldots$

2.5.2 Vettore caratteristico e argomento diagonale

Definition 2.5.1: Vettore caratteristico

Il **vettore caratteristico** del linguaggio $\mathcal{L}$ è una stringa binaria infinita $\chi_{\mathcal{L}}$ tale che la posizione i-esima vale 1 se $w_i \in \mathcal{L}$, e 0 altrimenti.

$$\chi_{\mathcal{L}} = \{0\ 1\ 1\ 0\ 0\ 1\ \cdots\}$$

Costruiamo ora una tabella infinita: sulle righe le MdT $M_1, M_2, \ldots$, sulle colonne le stringhe $w_1, w_2, \ldots$, e nella cella (i, j) scriviamo 1 se M_i accetta w_j, 0 altrimenti. Ogni riga è quindi il vettore caratteristico del linguaggio accettato da quella macchina.

	w_1	w_2	w_3	w_4	$\cdots$
M_1	**0**	1	0	0	$\cdots$
M_2	1	**1**	0	1	$\cdots$
M_3	0	0	**0**	1	$\cdots$
M_4	1	0	0	**1**	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Concentriamoci sulla **diagonale**: prendiamo il valore (i, i) per ogni i, ottenendo la stringa $D = \{0\ 1\ 0\ 1\ \cdots\}$. Definiamo $\bar{D}$ come il suo complemento bit a bit, $\bar{D} = \{1\ 0\ 1\ 0\ \cdots\}$.
$\bar{D}$ è interpretabile come vettore caratteristico di un linguaggio. Chiamiamo $\mathcal{L}_d$ il linguaggio il cui vettore caratteristico è $\bar{D}$:

$$\mathcal{L}_d = \{w_i \mid M_i \not\models w_i\}$$

ovvero $\mathcal{L}_d$ contiene la i-esima stringa se e solo se la i-esima macchina *non* la accetta.

Proposition 2.5.1

$\mathcal{L}_d \notin RE$

Dimostrazione: Supponiamo per assurdo che esista una MdT M_k tale che $\mathcal{L}(M_k) = \mathcal{L}_d$. Chiediamoci se M_k accetta w_k:

- Se $M_k \models w_k$: allora $w_k \in \mathcal{L}(M_k) = \mathcal{L}_d$, ma per definizione $w_k \in \mathcal{L}_d \iff M_k \not\models w_k$. Contraddizione.
- Se $M_k \not\models w_k$: allora $w_k \in \mathcal{L}_d$, quindi M_k dovrebbe accettare w_k. Contraddizione.

In entrambi i casi arriviamo a una contraddizione, quindi $\nexists$ una MdT che accetti $\mathcal{L}_d$, ovvero $\mathcal{L}_d \notin RE$

Note:

Perché esistono linguaggi indecidibili? Le MdT sono enumerabili, quindi sono tante quanti $\mathbb{N}$. I linguaggi invece sono tutti i possibili sottoinsiemi delle stringhe, quindi la loro cardinalità è quella di $\mathbb{R}$. Esistono semplicemente **più linguaggi che algoritmi**.

2.5.3 Proprietà sui linguaggi

Definition 2.5.2: Complemento di un linguaggio

Sia $\mathcal{L} \subseteq \Sigma^*$, il **complemento** di $\mathcal{L}$ è:

$$\overline{\mathcal{L}} = \Sigma^* \setminus \mathcal{L}$$

Theorem 2.5.1

Se $\mathcal{L} \in R \implies \overline{\mathcal{L}} \in R$

Dimostrazione: Siccome $\mathcal{L} \in R$, esiste $M_{\mathcal{L}}$ che decide $\mathcal{L}$ (risponde sì o no in tempo finito). Costruiamo $\overline{M}$ che si comporta esattamente come $M_{\mathcal{L}}$ ma inverte la risposta:

$$w \longrightarrow M_{\mathcal{L}} \xrightarrow{\text{sì}\to\text{no, no}\to\text{sì}} \overline{M}$$

Poiché $M_{\mathcal{L}}$ termina sempre, anche $\overline{M}$ termina sempre $\implies \overline{\mathcal{L}} \in R$.

Theorem 2.5.2

Se $\mathcal{L} \in RE \wedge \overline{\mathcal{L}} \in RE \implies \mathcal{L} \in R$

Dimostrazione: Siano $M_{\mathcal{L}}$ e $M_{\overline{\mathcal{L}}}$ le macchine che accettano rispettivamente $\mathcal{L}$ e $\overline{\mathcal{L}}$. Costruiamo M' che esegue $M_{\mathcal{L}}$ e $M_{\overline{\mathcal{L}}}$ **in parallelo** su un input w:

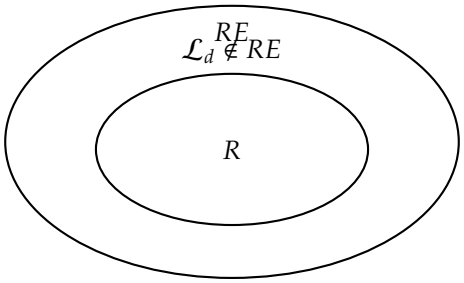
- Se risponde $M_{\mathcal{L}}$: $w \in \mathcal{L}$, accetta.
- Se risponde $M_{\overline{\mathcal{L}}}$: $w \notin \mathcal{L}$, rifiuta.

Poiché ogni stringa sta o in $\mathcal{L}$ o in $\overline{\mathcal{L}}$, una delle due macchine risponderà sempre in tempo finito $\implies M'$ è un decisore $\implies \mathcal{L} \in R$.

Note:

Questo risultato implica che RE **non è chiuso per complemento**: se lo fosse, ogni linguaggio in RE sarebbe automaticamente in R, e sappiamo che ciò non è vero.

Le classi R e RE formano quindi questa gerarchia stretta:



2.6 Linguaggio Universale e Problema della Fermata

Definition 2.6.1: Linguaggio Universale

Il linguaggio universale è l'insieme delle coppie macchina-stringa tali che la macchina M accetta w:

$$\mathcal{L}_u = \{(M, w) \mid M \models w\}$$

$\mathcal{L}_u \in RE$? Sì, perché esiste M_u che lo accetta: data la coppia (M, w), simula M su w e risponde sì se M accetta. Se M non accetta, M_u non termina, quindi nessuna garanzia sul "no".
$\mathcal{L}_u \in R$? No.

Theorem 2.6.1

$\mathcal{L}_u \notin R$

Dimostrazione: Supponiamo per assurdo che $\mathcal{L}_u \in R$, ovvero esiste $\bar{M}_u$ che *decide* $\mathcal{L}_u$. Costruiamo la macchina M':

1. Riceve w_i in input
2. Copia w_i per costruire la coppia (M_i, w_i)

3. Esegue $\bar{M}_u$ su (M_i, w_i) e **inverte** la risposta

Analizziamo $\mathcal{L}(M')$:

- M' risponde **sì** su $w_i \iff \bar{M}_u$ risponde no $\iff M_i \not\models w_i$
- M' risponde **no** su $w_i \iff \bar{M}_u$ risponde sì $\iff M_i \models w_i$

Quindi $\mathcal{L}(M') = \{w_i \mid M_i \not\models w_i\} = \mathcal{L}_d$. Ma M' termina sempre (poiché $\bar{M}_u$ termina sempre per ipotesi), quindi M' sarebbe un decisore per $\mathcal{L}_d$, il che implicherebbe $\mathcal{L}_d \in R \subseteq RE$. Contraddizione con $\mathcal{L}_d \notin RE$.

Note:

Vale anche che $\overline{\mathcal{L}_u} \notin RE$. Se così non fosse, avremmo $\mathcal{L}_u \in RE$ e $\overline{\mathcal{L}_u} \in RE$ contemporaneamente, il che implicherebbe $\mathcal{L}_u \in R$ per il teorema di chiusura — assurdo.

2.6.1 Il complemento di $\mathcal{L}_d$

Consideriamo il complemento di $\mathcal{L}_d$:

$$\bar{\mathcal{L}}_d = \{w_i \mid M_i \models w_i\}$$

Theorem 2.6.2

$\bar{\mathcal{L}}_d \in RE$

Dimostrazione: Costruiamo la macchina M: dato M_i in input, costruisce la coppia (M_i, w_i) e la passa a M_u. Se M_u accetta, risponde sì. Quindi:

- $M_i \in \bar{\mathcal{L}}_d \Rightarrow M$ risponde sì
- $M_i \notin \bar{\mathcal{L}}_d \Rightarrow M$ non risponde sì

Quindi $\bar{\mathcal{L}}_d \in RE$.

Note:

$\bar{\mathcal{L}}_d \notin R$: se lo fosse, anche $\mathcal{L}_d \in R$ per chiusura di R per complemento, da cui $\mathcal{L}_d \in RE$. Contraddizione.

2.6.2 Il Problema della Fermata

Definition 2.6.2: Problema della Fermata

$$HALT = \{(M, w) \mid M \text{ si arresta su } w\}$$

La differenza con $\mathcal{L}_u$ è sottile: in $\mathcal{L}_u$ la macchina deve fermarsi *e accettare*; in $HALT$ basta che si fermi, indipendentemente dal risultato.

Theorem 2.6.3

$HALT \in RE$

Dimostrazione: La macchina M_h accetta $HALT$: simula M su w tramite M_u, risponde sì se M si arresta. Se M non si arresta, M_h non termina. Quindi:

- $(M, w) \in HALT \Rightarrow M_h$ risponde sì
- $(M, w) \notin HALT \Rightarrow M_h$ non risponde sì

Theorem 2.6.4

$HALT \notin R$

Dimostrazione: Supponiamo per assurdo che $HALT \in R$, ovvero esiste M_h^* che decide $HALT$. Costruiamo M' su input (M, w):

1. Esegue M_h^* su (M, w)
2. Se M_h^* risponde **no**: M non si ferma su w, quindi M' risponde no
3. Se M_h^* risponde **sì**: M si ferma su w, quindi M' simula M su w tramite M_u e ne usa la risposta

In questo modo M' decide $\mathcal{L}_u$, poiché M_h^* garantisce l'arresto anche sul "no" (loop $\rightarrow$ no immediato). Quindi $\mathcal{L}_u \in R$. Contraddizione.

2.6.3 La variante $HALT_\varepsilon$

Definition 2.6.3: $HALT_\varepsilon$

$$HALT_\varepsilon = \{M \mid M \text{ si arresta su } \varepsilon\}$$

Insieme delle MdT che si arrestano quando ricevono la stringa vuota in input.

Theorem 2.6.5

$HALT_\varepsilon \in RE$

Dimostrazione: La macchina $M_{h\varepsilon}$ prende in input M, simula M su ε tramite M_u, e risponde sì se M si arresta. Quindi:

- $M \in HALT_\varepsilon \Rightarrow M_{h\varepsilon}$ risponde sì
- $M \notin HALT_\varepsilon \Rightarrow M_{h\varepsilon}$ non risponde sì

Theorem 2.6.6

$HALT_\varepsilon \notin R$

Dimostrazione: Supponiamo per assurdo che esista $M_{h\varepsilon}^*$ che decide $HALT_\varepsilon$. Costruiamo $\tilde{M}_w$: una macchina che ignora il proprio input, scrive w sul nastro e simula M su w. Costruiamo quindi M' su input (M, w):

1. Costruisce $\tilde{M}_w$ a partire da M e w
2. Esegue $M_{h\varepsilon}^*$ su $\tilde{M}_w$

Poiché $\tilde{M}_w$ su ε si comporta esattamente come M su w, abbiamo:

- M' risponde sì $\iff$ $M_{h\varepsilon}^*$ dice sì su $\tilde{M}_w$ $\iff$ $\tilde{M}_w$ si arresta su ε $\iff$ M si arresta su w
- M' risponde no altrimenti

Quindi M' decide $HALT \Rightarrow HALT \in R$. Contraddizione.

Chapter 3

Riduzioni

Le dimostrazioni dei teoremi del capitolo precedente avevano tutte un approccio simile: supponiamo per assurdo che un certo linguaggio sia decidibile, costruiamo una macchina ausiliaria sfruttando il decisore assunto esistere, e arriviamo a una contraddizione. Queste erano istanziazioni di un principio molto più generale: la **riduzione tra linguaggi**.

Definition 3.0.1: Riduzione

Siano A, B due linguaggi. La funzione f è una **riduzione** da A a B se per ogni stringa w:

$$w \in A \iff f(w) \in B$$

e inoltre f è calcolabile. Quando esiste una riduzione da A a B scriviamo $A \leqslant B$.

Note:

Intuitivamente, $A \leqslant B$ significa che A non è più difficile di B: saper risolvere B implica saper risolvere A, applicando prima f.

Per definire formalmente la calcolabilità di f è necessario introdurre un nuovo tipo di macchina.

Definition 3.0.2: Trasduttore

Un **trasduttore** è una Macchina di Turing che calcola funzioni, caratterizzata da tre nastri:

1. **Input** — sola lettura
2. **Worktape** — lettura e scrittura
3. **Output** — sola scrittura

Un trasduttore M *calcola* la funzione f se, per ogni stringa w, quando M esegue su w e si arresta, lascia $f(w)$ scritto sul nastro di output.

Note:

Distinguiamo trasduttori (calcolatori di funzioni) dalle MdT classiche (decisori). Una **funzione calcolabile** è una funzione per cui esiste un trasduttore che la calcola e che si arresta sempre in tempo finito su ogni input.

3.1 Teorema sulle riduzioni

Theorem 3.1.1

Siano A, B due linguaggi tali che $A \leqslant B$. Allora:

1. Se $A \notin R \Rightarrow B \notin R$

2. Se $A \notin RE \Rightarrow B \notin RE$

Dimostrazione: **Caso 1.** Supponiamo per assurdo che $A \notin R$ ma $B \in R$. Allora esiste M_B che decide B. Costruiamo M_A su input w:

1. Calcola $f(w)$ tramite il trasduttore per f
2. Esegue M_B su $f(w)$
3. Risponde come M_B

Il trasduttore termina sempre per definizione di funzione calcolabile, e M_B termina sempre perché decide B. La risposta è corretta poiché $w \in A \iff f(w) \in B$. Quindi M_A decide $A \Rightarrow A \in R$. Contraddizione.
Caso 2. Supponiamo per assurdo che $A \notin RE$ ma $B \in RE$. Allora esiste M_B che accetta B. Costruiamo M_A con la stessa pipeline di prima: calcola $f(w)$ e poi esegue M_B su $f(w)$. Allora:

- $w \in A \Rightarrow f(w) \in B \Rightarrow M_B$ accetta $\Rightarrow M_A$ accetta
- $w \notin A \Rightarrow f(w) \notin B \Rightarrow M_B$ non accetta $\Rightarrow M_A$ non accetta

Quindi M_A accetta $A \Rightarrow A \in RE$. Contraddizione.

Corollary 3.1.1

Siano A, B due linguaggi tali che $A \leqslant B$. Allora:

$$B \in R \Rightarrow A \in R \qquad B \in RE \Rightarrow A \in RE$$

Dimostrazione: Segue direttamente per contronominale dal teorema precedente: $(A \notin R \Rightarrow B \notin R) \equiv (B \in R \Rightarrow A \in R)$.

Note:

Nella pratica usiamo sempre il corollario in questa direzione: vogliamo dimostrare che B è difficile. Partiamo da un A noto essere indecidibile, mostriamo $A \leqslant B$, e concludiamo che B non può essere decidibile — altrimenti lo sarebbe anche A.

3.2 Esempio: $\mathcal{L}_{ne}$

Definition 3.2.1: $\mathcal{L}_{ne}$

$$\mathcal{L}_{ne} = \{M_i \mid \mathcal{L}(M_i) \neq \emptyset\}$$

L'insieme delle MdT che accettano almeno una stringa.

Theorem 3.2.1

$\mathcal{L}_{ne} \in RE$

Dimostrazione: Costruiamo la macchina M_{ne} che accetta $\mathcal{L}_{ne}$. Data M_i in input, M_{ne} non sa a priori quale stringa M_i potrebbe accettare, quindi la **indovina** non deterministicamente (design pattern *guess & check*):

1. Indovina non deterministicamente una stringa w
2. Simula M_i su w tramite M_u
3. Se M_i accetta w, risponde sì

Se $\mathcal{L}(M_i) \neq \emptyset$, esiste almeno una stringa w^* accettata da M_i: un ramo non deterministico la indovinerà e risponderà sì in tempo finito. Non diamo garanzie sul no, ma per RE non è richiesto.

Theorem 3.2.2

$\mathcal{L}_{ne} \notin R$

Dimostrazione: Mostriamo la riduzione $\mathcal{L}_u \leqslant \mathcal{L}_{ne}$. Dobbiamo costruire una funzione f calcolabile tale che:

$$(M, w) \in \mathcal{L}_u \iff f(M, w) \in \mathcal{L}_{ne}$$

La funzione f, data la coppia (M, w), costruisce una nuova macchina M' definita come segue: M' **ignora il proprio input** x, scrive w sul nastro, e simula M su w.
Verifichiamo la correttezza di f in entrambe le direzioni:
($\Rightarrow$) Supponiamo $(M, w) \in \mathcal{L}_u$, ovvero $M \models w$. Allora M', su qualsiasi input x, ignora x, simula M su w, e accetta — poiché M accetta w. Quindi $\mathcal{L}(M') = \Sigma^* \neq \emptyset$, da cui $M' \in \mathcal{L}_{ne}$.
($\Leftarrow$) Supponiamo $(M, w) \notin \mathcal{L}_u$, ovvero $M \not\models w$. Allora M', su qualsiasi input x, simula M su w e non accetta mai. Quindi $\mathcal{L}(M') = \emptyset$, da cui $M' \notin \mathcal{L}_{ne}$.
La riduzione è corretta. Poiché $\mathcal{L}_u \notin R$ e $\mathcal{L}_u \leqslant \mathcal{L}_{ne}$, per il teorema sulle riduzioni segue $\mathcal{L}_{ne} \notin R$.

Chapter 4

Teorema di Rice

Chapter 5

Complessita Strutturale

Chapter 6

Complessita Spaziale

Chapter 7

Oracoli e Classi Funzionali